Text Mining and Document Intelligence System for News Analytics

Information Retrieval (AIMLCZG537/DSECLZG537) S1-26 - Assignment 2

Group ID: G35

#	Registration number	Name
1	2025AG05062	LAKSHYA KANCHAN
2	2025AG05772	SHREYANSH SHARMA
3	2025AG05689	VINAY PRATAP SINGH BHADAURIA
4	2025AG05914	NAIR ABHIJITH VIJAYAKUMAR
5	2025AG05076	ABHISHEK GHOSH

1. Selected business context

We take the role of a text-mining team at a media-monitoring and business-research firm that processes a continuous stream of news articles.

Analysts at such a firm (and the communications, investor-relations and policy teams who buy its services) need to:

- **route** each incoming article to the right desk (business, politics, tech, entertainment, sport) - *classification*;
- **discover** topical groupings without waiting for labels - *clustering*;
- **recognise** syndicated or re-published copies so coverage is not counted twice - *duplicate detection*;
- **pull up related coverage** while researching a story - *recommendation*.

The system is a Streamlit application (dashboard, data ingestion, preprocessing configuration, text-mining hub, recommendation panel, performance analytics, inferences) backed by a hand-written Python package.

2. Dataset description

The BBC News corpus contains 2,225 documents and 5.03 MB of raw text across five labelled desks, including duplicates and noise.

- **Source:** BBC News full-text corpus, 2004-2005 (Greene and Cunningham, ICML 2006; mlg.ucd.ie/datasets/bbc.html). The first line of each file is the headline.
- **Structure:** `data/documents.csv` (doc_id, text) holds only text; `data/metadata.csv` (doc_id, title, category, source, period, filename, n_chars, n_words) holds metadata. IDs look like BUS-001.
- **Size:** 5,033,302 characters, 842,910 words; 378.8 words per document on average (median 326, range 84-4,428).
- **Labels:** business 510, entertainment 386, politics 417, sport 511, tech 401 - moderately balanced.
- **Noise:** 2,116 documents contain numbers, 771 currency symbols, 1,921 quotation marks; 15.7% of tokens are capitalised (names, sentence starts).

- **Duplicates:** 137 exact-duplicate pairs plus 26 near-duplicate pairs (cosine >= 0.9); de-duplication removes 162 documents (79 of them tech), leaving 2,063. No duplicate pair crosses categories.

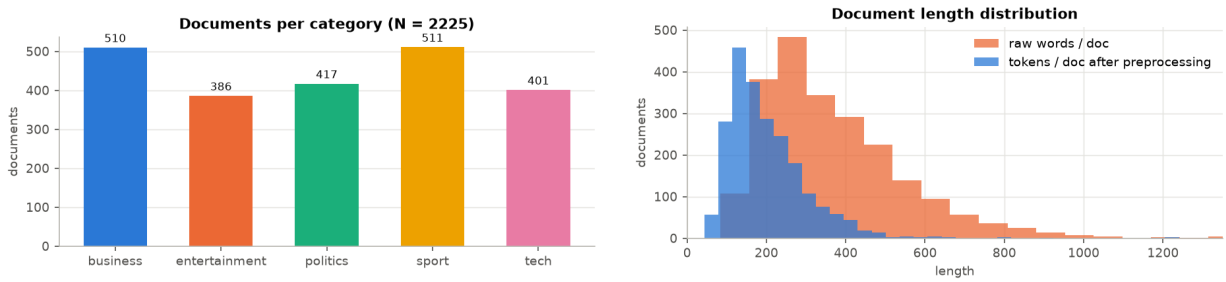


Figure 1. *Left: documents per category. Right: document length before (raw words) and after preprocessing (tokens) - preprocessing roughly halves each document.*

3. Implementation approach

All core IR algorithms are written by hand in the textmining/ package; the Streamlit app and the experiment script use the same implementation for the app and report results.

Pipeline. ingest (CSV / JSON / ZIP of .txt) -> preprocess -> inverted index -> TF-IDF -> { Naive Bayes | K-Means + Purity/RI | cosine Top-K recommender | duplicate detector } -> Streamlit visualisations. `experiments.py` (seed 42) writes every number in this report to `report/results.json`.

No built-in implementations. The inverted index, TF-IDF, cosine similarity, Naive Bayes, K-Means, Purity and Rand Index are hand-implemented; scikit-learn is not used (not even in requirements.txt). numpy is used only as an array container, NLTK only supplies the Porter stemmer, WordNet lemmatiser and stop-word list, and PCA is used only to draw the 2-D scatter plot.

Component	Formula / procedure
TF-IDF (ltc)	`w(t,d) = (1 + log10 tf(t,d)) * log10(N / df_t)`
L2 normalisation	`w'(t,d) = w(t,d) / sqrt( sum_t' w(t',d)^2 )`
Cosine similarity	`cos(a,b) = a.b / (|a| |b|) = a.b (rows are unit length)`
Multinomial NB	`P(t|c) = (T_ct + 1) / (sum_t' T_ct' + |V|); c* = argmax_c log P(c) + sum_t tf(t,d) log P(t|c)`
K-Means	k-means++ init -> assign each doc to nearest centroid (squared Euclidean) -> centroid = mean of members -> stop when no document moves, max shift < 1e-4, or 100 iterations; best SSE of 5 restarts
Purity	`purity = (1/N) sum_k max_j |w_k AND c_j|`
Rand Index	`RI = (TP + TN) / C(N,2); TP = sum_kj C(n_kj,2), TP+FP = sum_k C(|w_k|,2), TP+FN = sum_j C(|c_j|,2)`

Table 1. *Formulas implemented (w_k = cluster k, c_j = class j, C(n,2) = number of pairs).*

Evaluation protocol. Naive Bayes uses a stratified 80/20 split (seed 42; 1,781 train / 444 test) and builds its vocabulary from training documents only. Clustering, recommendation and duplicate detection never see labels; labels are used only afterwards to compute Purity, Rand Index and Precision@K.

4. Preprocessing and feature choices

Lowercasing, punctuation/number removal, stop-word removal and Porter stemming reduce the vocabulary by 44.1% and the token count by 53.0%, leaving fewer features focused on article content.

Cumulative step	Tokens	Vocabulary	Avg tokens / doc
Raw tokens (no normalisation)	961,301	37,981	432.0
Lowercase + drop punctuation	843,375	33,096	379.0
+ drop numbers, min length 2	800,959	31,133	360.0
+ stop-word removal	451,770	30,998	203.0
+ Porter stemming (baseline)	451,770	21,238	203.0

Table 2. *Effect of each preprocessing step on the full corpus.*

- **Baseline configuration:** markup stripping, lowercasing, regex tokenisation, drop punctuation and numbers, minimum token length 2, NLTK English stop-words, Porter stemming, unigrams. Lemmatisation and n-grams are switchable in the app and are compared in the ablation (Section 9).
- **Stop-words** remove 43.6% of tokens but almost no vocabulary; **stemming** then merges word forms (30,998 -> 21,238 types).
- **TF-IDF features:** ltc weighting with L2 normalisation; terms with df < 2 (9,031 terms) or df > 90% of N are pruned and the 5,000 highest-df terms kept, giving a 2225 x 5000 matrix (2.47% non-zero; every row has norm 1.0).
- **Worked example:** term 'aol' in BUS-001: tf = 7, df = 15, weight 1.8451 x idf 2.1712 = 4.0061, divided by the document length 18.2874 = 0.2191 (identical to the matrix entry).

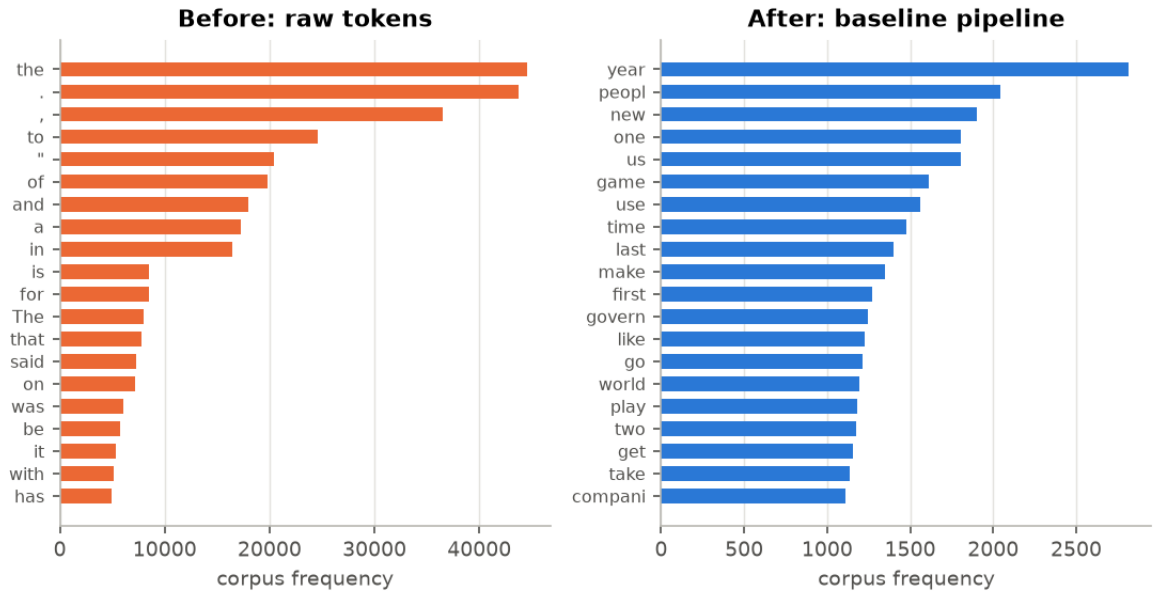


Figure 2. *Top-20 terms before (raw tokens) and after the baseline pipeline: function words and punctuation are replaced by content stems such as 'year', 'peopl', 'game' and 'govern'.*

5. Classification results (Naive Bayes)

The hand-written multinomial Naive Bayes classifier labels 96.2% of the 444 test articles correctly (427 right, 17 wrong), with macro-F1 0.960.

Class	Precision	Recall	F1	Support
business	0.980	0.951	0.965	102
entertainment	0.973	0.935	0.954	77
politics	0.941	0.952	0.946	83
sport	0.990	0.980	0.985	102
tech	0.919	0.988	0.952	80
Accuracy			**0.962**	444
Macro avg	0.960	0.961	0.960	444
Weighted avg	0.963	0.962	0.962	444

Table 3. *Per-class and averaged metrics on the held-out test set.*

doc_id	Title	Actual	Predicted	P(c|d)
BUS-054	Hyundai to build new India plant	business	business	1.000
BUS-133	Salary scandal in Cameroon	business	politics (x)	0.976
BUS-213	US bank boss hails 'genius' Smith	business	politics (x)	1.000
BUS-226	McDonald's to sponsor MTV show	business	entertainment (x)	0.999
BUS-315	S&N; extends Indian beer venture	business	business	1.000
BUS-384	Weak end-of-year sales hit Next	business	business	1.000
ENT-096	Singer's film to show at festival	entertainment	entertainment	1.000
ENT-153	OutKast win at MTV Europe Awards	entertainment	entertainment	1.000

Table 4. *Sample of predicted labels ((x) = misclassified). All 444 predictions are in* `report/nb_test_predictions.csv` *and in the app.*

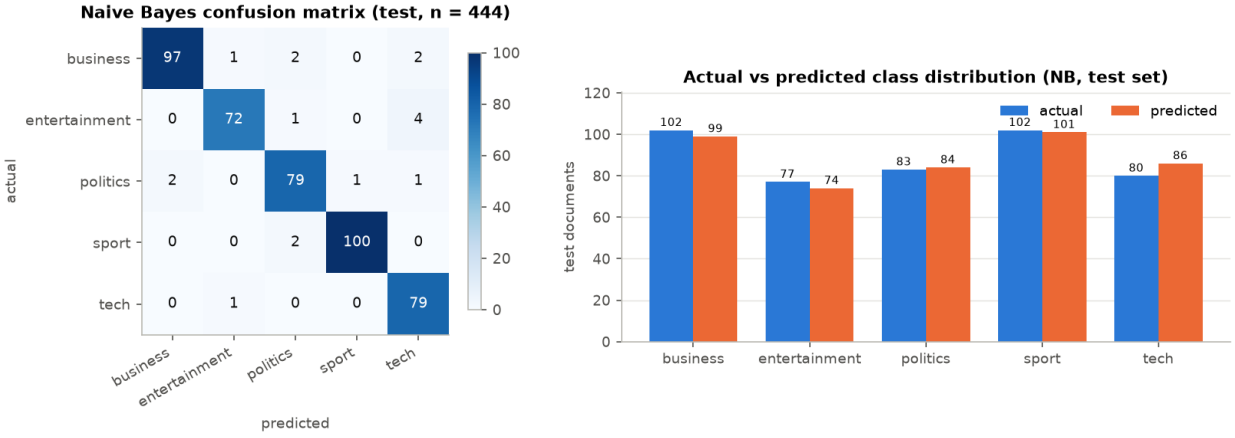


Figure 3. *Left: confusion matrix (rows = actual, columns = predicted). Right: actual vs predicted class distribution.*

Errors are mostly cross-desk stories (e.g. a salary scandal labelled politics, a media company's games deal labelled tech); tech has the lowest precision (0.919) because it attracts 7 false positives. **Leakage note:** 38 test articles have an exact copy in the training set and are all classified correctly (1.000 vs 0.958 on the rest) - see Q2.

6. Clustering results (K-Means)

Hand-written K-Means (k = 5) groups articles by the five desks without using labels: purity 0.967, Rand Index 0.975.

Configuration. k = 5, k-means++ initialisation, 5 restarts (best SSE kept), squared Euclidean distance on L2-normalised TF-IDF rows, tol 0.0001, max 100 iterations, seed 42. The best run stopped because it **no document changed cluster at iteration 31** (final SSE 2072.7656).

Iteration	Docs moved	Max centroid shift	SSE	Cluster sizes
1	2,225	0.9598	2108.13	472 / 406 / 474 / 524 / 349
2	646	0.0753	2090.82	376 / 559 / 376 / 528 / 386
5	44	0.0273	2084.37	357 / 717 / 364 / 500 / 287
21	127	0.0488	2076.21	524 / 448 / 388 / 512 / 353
28	5	0.0053	2072.78	435 / 392 / 387 / 513 / 498
31	0	0.0000	2072.77	432 / 392 / 387 / 513 / 501

Table 5. *Condensed iteration log of the best run (31 iterations; the full log is shown in the app). SSE falls monotonically; the increase in document moves around iteration 21 occurs as an oversized cluster splits before convergence.*

Metric	Value	Pair counts	Value
Purity	0.967	TP (same cluster, same class)	470,082
Rand Index	0.975	FP (same cluster, different class)	30,919
SSE	2072.8	FN (different cluster, same class)	31,259
Documents	2,225	TN (different cluster, different class)	1,941,940

Table 6. *Clustering evaluation against the true desks.*

Verification. The fast contingency-table Rand Index equals an explicit O(N^2) pair loop on a 300-document sample (0.9729 vs 0.9729). **Seed stability:** over 5 seeds purity is 0.963 +/- 0.004 with 5 restarts, but ranges 0.782-0.967 with a single initialisation.

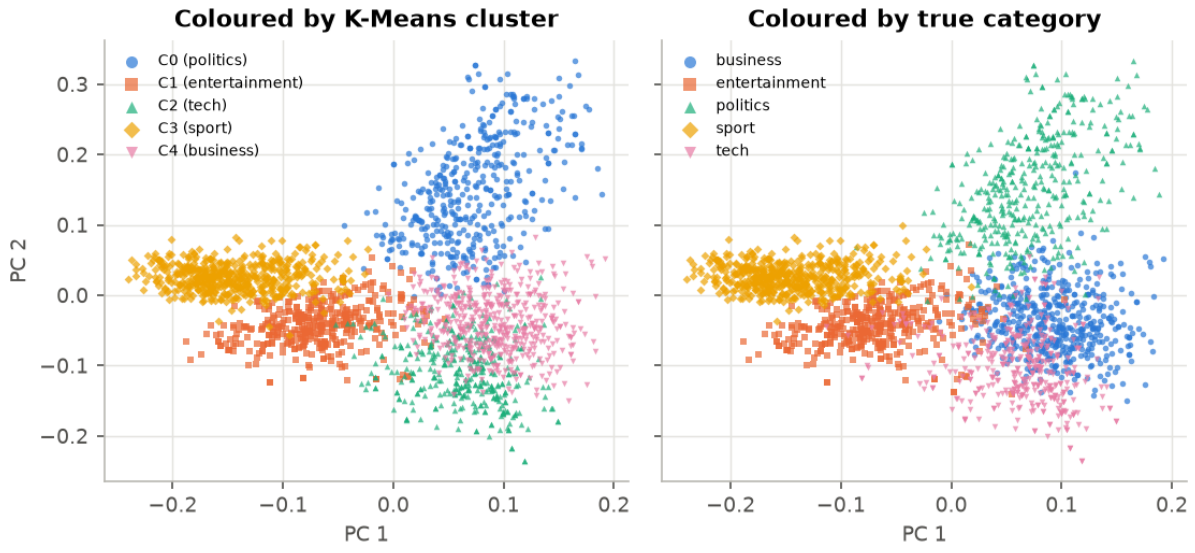
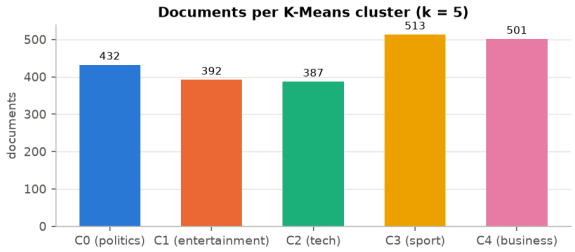


Figure 4. *2-D PCA projection of all documents, coloured by K-Means cluster (left) and by true category (right). PCA is used for display only; clustering runs in the full 5,000-dimensional space.*



Cluster	Majority	Size	Share	Top terms
C0	politics	432	94.0%	elect, labour, parti
C1	entertainment	392	95.7%	film, star, award
C2	tech	387	96.4%	user, technolog, use
C3	sport	513	99.2%	game, win, match
C4	business	501	97.4%	market, growth, compani

Figure 5. *Left: documents per cluster. Right: cluster assignments vs true labels (majority label names the cluster; full membership in* `report/kmeans_membership.csv`*).*

7. Comparison of classification and clustering results

Both methods agree with the editorial desks about 96% of the time, but they answer different questions and their numbers are not directly interchangeable.

	Naive Bayes (supervised)	K-Means (unsupervised)
Uses labels	Yes - trained on 1,781 labelled docs	No - labels only for evaluation
Evaluated on	444 held-out test docs	All 2,225 docs
Main metrics	Accuracy 0.962; macro P/R/F1 0.960 / 0.961 / 0.960	Purity 0.967; Rand Index 0.975
5-seed mean	Accuracy 0.970 +/- 0.009	Purity 0.963; RI 0.972
Output	A desk label + posterior for every new article	Unnamed groups; names come from majority labels / top terms

Table 7. *Naive Bayes vs K-Means.*

Naive Bayes is **supervised**: it learns the desk boundaries from labelled examples and is judged on unseen documents, so its accuracy estimates generalisation and it can route new articles directly. K-Means is **unsupervised**: it only groups similar documents, and purity (0.967) equals the accuracy of mapping each cluster to its majority class - computed on the same data that was clustered, after looking at the labels. The Rand Index is inflated by true negatives (78.5% of all pairs are 'different cluster, different class'). K-Means also needs k in advance and several restarts to be stable, whereas NB varies only with the split. The similar results suggest that vocabulary separates the five desks well.

8. Recommendation results

Content-based Top-K recommendation by TF-IDF cosine similarity: on average 93.3% of the top-5 results share the query's desk.

Approach. The query (a selected document or free text, preprocessed with the same pipeline) is a TF-IDF vector q; every other document d is scored by `score(q,d) = cos(q,d) = q.d` and the K highest are returned with their metadata and the shared terms that contribute most to the score. Documents with cosine >= 0.9 to the query (near-duplicates) can be hidden.

#	Score	doc_id	Title	Category	Source	Shared terms
1	0.340	POL-340	Lib Dems unveil women's manifesto	politics	BBC News	matern, carer, hewitt, women
2	0.292	POL-128	Brown shrugs off economy fears	politics	BBC News	matern, prebudget, mother, taxpay
3	0.264	POL-003	Hewitt decries 'career sexism'	politics	BBC News	matern, hewitt, women, patricia
4	0.219	BUS-382	Ban on forced retirement under 65	business	BBC News	hewitt, patricia, commerc, older
5	0.200	POL-390	Child access law shake-up planned	politics	BBC News	recycl, father, parent, outlin

Table 8. *Top-5 recommendations for POL-001 'Labour plans maternity pay rise' (politics). Period: 2004-2005 for all.*

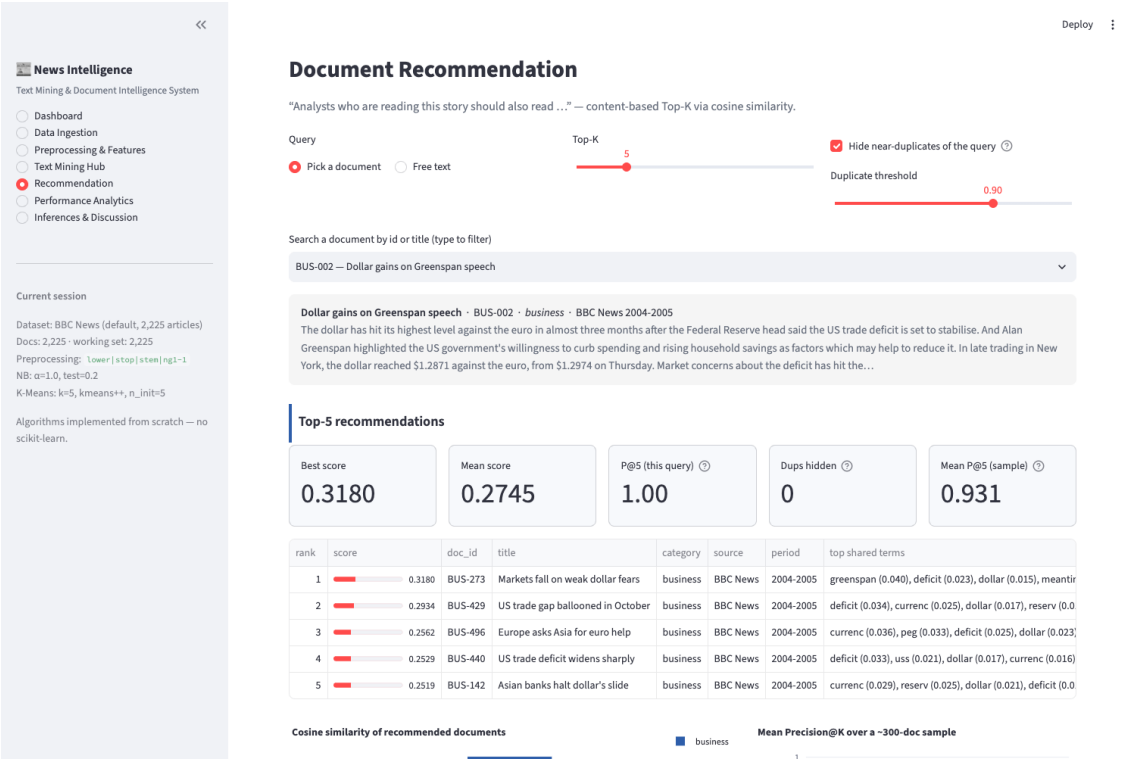


Figure 6. *Recommendation panel of the Streamlit app: query selection, Top-K slider, results with scores, metadata and shared-term explanations.*

Quality. Using 'same category' as a relevance proxy, P@5 = 0.933 and P@10 = 0.920 (sport best at 0.986, entertainment lowest at 0.902). A free-text query ('mobile phone operators launch faster broadband and music download services') returns five tech articles led by 'Mobile gig aims to rock 3G' (0.242).

Findings and limitations.

- Results are explainable: shared stems such as matern, carer, hewitt show why POL-340 is ranked first; the cross-desk hit BUS-382 is still topically relevant (employment policy), so the category proxy understates quality.
- Duplicates occupy recommendation slots: 323 documents have a near-copy in their top-5 (e.g. BUS-007's #1 result is a verbatim copy, score 1.000). The hide-duplicates option removes these results.
- Bag-of-words cannot match synonyms or paraphrases, and short or generic queries (e.g. TEC-001 on election ink) drift to other desks because a few rare shared words dominate.
- No user interaction data exists, so there is no personalisation, popularity or recency signal.

9. Inferences and limitations

We answer the five compulsory questions using ablation tests (each configuration re-runs the full pipeline with and without de-duplication).

Configuration	Vocab (raw)	NB acc	NB macro-F1	Purity (5-seed)	RI (seed 42)	P@5
Baseline (stop + stem)	21,238	0.962	0.960	0.959 +/- 0.009	0.975	0.933
Baseline + dedup	21,227	0.983	0.982	0.872 +/- 0.074	0.886	0.917
No stop-word removal	21,332	0.960	0.958	0.911 +/- 0.087	0.975	0.936
Lemmatization	27,572	0.966	0.965	0.915 +/- 0.075	0.966	0.938
No normalizer	30,998	0.966	0.965	0.889 +/- 0.085	0.962	0.936
Uni+bigram	321,720	0.964	0.962	0.859 +/- 0.079	0.965	0.946
No lowercasing, no stemming	35,047	0.969	0.967	0.889 +/- 0.083	0.958	0.949

Table 9. *Ablation. Purity (5-seed) = mean +/- std over 5 seeds with fewer restarts than the main run, so it is slightly below Section 6.*

Q1. How preprocessing choices affected the results

- **Vocabulary:** the baseline has 21,238 types; dropping stemming raises it to 30,998, lemmatisation gives 27,572, no lowercasing and no stemming 35,047, and bigrams increase it to 321,720. (Disabling lowercasing alone changes nothing because the Porter stemmer lowercases its output.)
- **Classification** changes little: NB accuracy stays within 0.960-0.969, smaller than the spread across random splits (0.970 +/- 0.009). Keeping stop-words is slightly worse (0.960); the benefit of the baseline is a much smaller model.
- **Clustering** is sensitive: the baseline has the highest seed-averaged purity (0.959 +/- 0.009) versus 0.911 without stop-word removal, 0.889 without stemming and 0.859 with bigrams. Larger vocabularies spread the 5,000 features over redundant variants and create more local optima.
- **Recommendation** prefers specific features: P@5 rises from 0.933 (baseline) to 0.946 with bigrams and 0.949 with case-sensitive unstemmed tokens, because exact rare-term overlap matters for nearest neighbours. Stop-words + stemming is the best overall compromise.

Q2. Effect of duplicates and how they were detected and handled

- **Detection:** exact = MD5 of lower-cased, whitespace-collapsed text (137 pairs); near = TF-IDF cosine >= 0.9 confirmed by word 5-shingle Jaccard (26 more pairs, mean Jaccard 0.8867). Union-find merges pairs; 162 redundant documents can be dropped, or just flagged/hidden in the app.
- **Classification:** 38 of 444 test documents have a copy in training and score 1.000 accuracy (vs 0.958); this reflects memorisation of copies. After de-duplication leakage is 0 and accuracy is 0.983 (5 splits: 0.976). Only the de-duplicated result estimates accuracy without copy leakage.
- **Clustering:** copies have distance 0, always share a cluster and add guaranteed TP pairs and double weight to centroids. Removing them made K-Means less stable: seed-42 purity 0.967 -> 0.792; 5-seed mean 0.963 -> 0.927. With 20 restarts it returns to 0.961, so the drop is an optimisation effect.
- **Recommendation:** 323 result lists contain a near-copy, wasting slots and inflating P@5 (0.933 vs 0.931 with duplicates hidden).

Q3. Recommendation approach: content-based vs collaborative

We use **content-based** recommendation (TF-IDF cosine, Section 8) because the corpus has full article text but no user-document interaction data, so collaborative filtering would have nothing to learn from.

- **Content-based** needs only item content (text, metadata). It works for brand-new articles and new users, is explainable through shared terms, but over-specialises and cannot judge quality. Preferable when the catalogue changes fast (news), users are few, or transparency is required.
- **Collaborative** needs a user-item interaction matrix (reads, clicks, ratings, saves) from many users. It finds co-read items that may not be obvious from content and captures preferences, but has cold-start and sparsity problems. Preferable with many active users and a stable catalogue.
- A production system would be **hybrid**: content-based for fresh stories, blended with collaborative signals once analyst reading logs exist.

Q4. How the techniques help domain experts decide

- **Text mining** measures the feed (volume per desk, key vocabulary) and identifies 162 redundant copies (7.3%), so coverage reports avoid counting the same story twice.
- **Classification** routes articles to desks with 96.2% accuracy; low-confidence or cross-desk predictions can go to a human review queue.
- **Clustering** finds groupings without labels (purity 0.967) - useful for an unlabelled client archive or for spotting emerging sub-themes with a larger k.
- **Recommendation** provides related coverage for a story, with duplicates removed and shared terms explaining each result. The workflow is ingest -> de-duplicate -> route -> explore themes -> brief. Measurements at each step help analysts assess reliability.

Q5. Naive Bayes vs K-Means: findings, limitations, improvements

NB reaches accuracy 0.962 and macro-F1 0.960 on held-out data; K-Means reaches purity 0.967 and RI 0.975 without labels (5-seed mean purity 0.963). Vocabulary separates the desks well; preprocessing changes the feature space more than NB accuracy; duplicates inflate NB accuracy through leakage and inflate P@5; single-seed K-Means results are unreliable without restarts.

Limitations and improvements for real-world deployment

- **Data:** one clean, balanced 2004-05 source with five desks; a live feed is multi-source, multilingual and drifts - retrain periodically and use time-based splits.
- **Features:** bag-of-words ignores synonyms and word order - add LSA or sentence embeddings; use sparse matrices and an approximate nearest-neighbour index to scale beyond tens of thousands of documents.
- **Classification:** NB posteriors are over-confident (~1.0 even on errors) - calibrate or use logistic regression, allow multi-label output, and de-duplicate before splitting.
- **Clustering:** k must be fixed and runs hit local optima - use more restarts, spherical K-Means on cosine, and choose k by silhouette or stability.
- **Recommendation:** same-category is only a relevance proxy - collect analyst judgements; add MinHash-LSH duplicate collapsing, diversity (MMR), recency and collaborative signals.

10. Evidence of execution in the BITS Virtual Lab

Screenshots of the working Streamlit front end launched with `streamlit run app.py`; the recommendation panel is shown in Section 8.

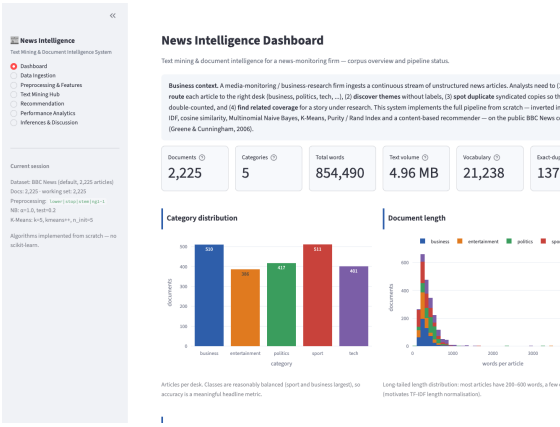


Figure 7. *Dashboard and corpus overview*

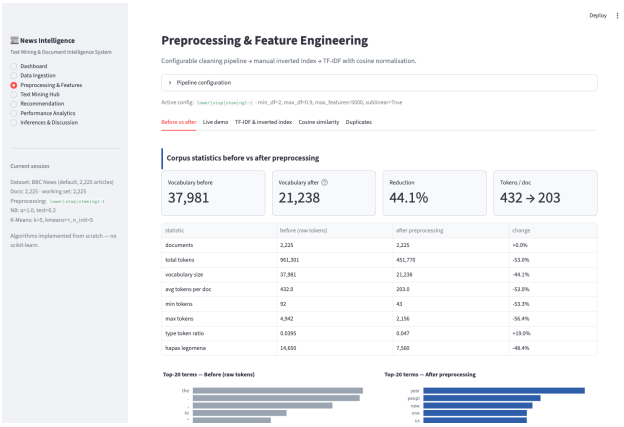


Figure 8. *Preprocessing: before/after distributions*

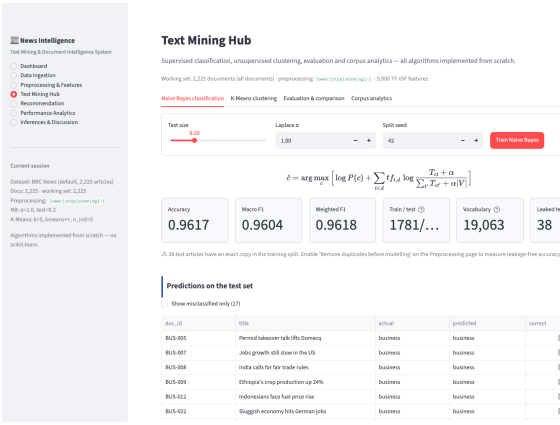


Figure 9. *Naive Bayes classification*

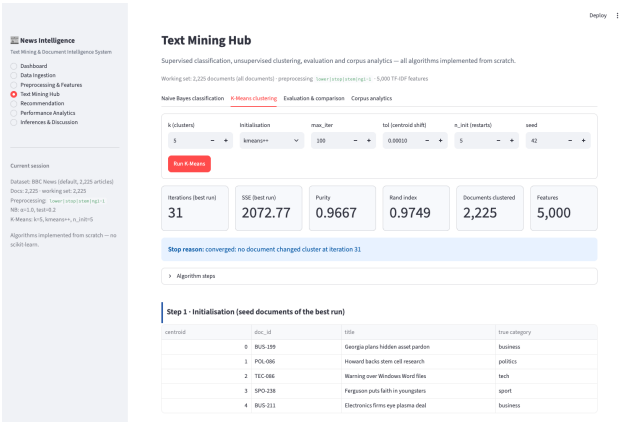


Figure 10. *K-Means clustering steps*

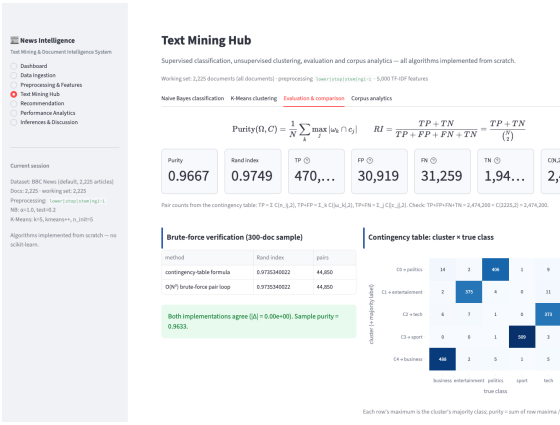


Figure 11. *Evaluation and comparison*

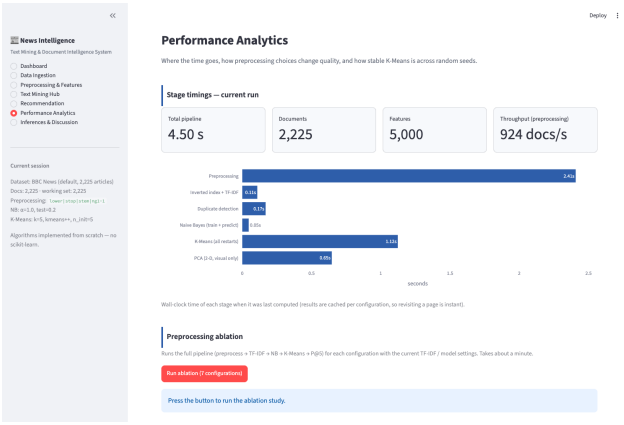


Figure 12. *Performance analytics*

Appendix: how to run and references

```
pip install -r requirements.txt
streamlit run app.py
python experiments.py # regenerates report/results.json and figures (seed 42)
python report/build_report.py # rebuilds this PDF
```


- D. Greene, P. Cunningham. Practical Solutions to the Problem of Diagonal Dominance in Kernel Document Clustering. ICML 2006 (BBC dataset).
- C. D. Manning, P. Raghavan, H. Schuetze. Introduction to Information Retrieval. Cambridge University Press, 2008 (Ch. 6, 13, 16).
- M. F. Porter. An algorithm for suffix stripping. Program 14(3), 1980.
- D. Arthur, S. Vassilvitskii. k-means++: The Advantages of Careful Seeding. SODA 2007.
- W. M. Rand. Objective criteria for the evaluation of clustering methods. JASA 66(336), 1971.